The Cloud Developer Workbook
100 Hands on exercises to build your aws skills

Summary by
Sadaf Iqbal, Muhammad Suleman

Exercise 1: Setting Up Your AWS Account

Concepts:

- AWS Free Tier gives free usage hours for new accounts.
- Security of the root account is critical.
- IAM allows setting password policies and managing users.

Tasks & Step-by-Step Solutions:

1. **Set up a new AWS account:**
    - Go to AWS Signup
    - Provide email, password, and account info
    - Choose “Personal” or “Professional” plan (Personal is fine for learning)
2. **Activate Multi-Factor Authentication (MFA) for the root account:**
    - Sign in as root user → Click your account name → “Security Credentials”
    - Select “Activate MFA” → Choose Virtual MFA Device → Scan QR code with Authenticator app → Enter 2 consecutive codes to activate
3. **Set a password policy in IAM:**
    - Go to IAM console → Account settings → Password policy → Require:
        - Minimum 8 characters
        - Uppercase, lowercase, numbers, symbols
        - Password expiration (optional)
    - Save changes

Consider Questions & Suggested Answers:

1. **Why should your root account be extra secure?**
    - Root has full access; if compromised, all resources, billing, and data are at risk.
2. **What do you hope to achieve by completing these 100 exercises?**
    - Gain hands-on experience with AWS services
    - Learn practical cloud development skills
3. **What do you want to do with AWS?**
    - Build and deploy cloud applications
    - Manage scalable and secure cloud infrastructure

Exercise 2: Your First IAM User

Concepts:

- Root user = full access, risky for everyday tasks.
- IAM users = safer way to access AWS.
- Groups allow easier management of permissions for multiple users.
- Policies define permissions (what actions a user/group can perform).

Tasks & Step-by-Step Solutions:

1. **Create a new IAM group "Admins"**
    - Go to AWS → IAM → Groups → Create New Group → Name: `Admins`
    - Attach policy → `AdministratorAccess` → Create group
2. **Create a new IAM user with your name**
    - IAM → Users → Add User → Username: Your Name
    - Select “Password – AWS Management Console access”
    - Set password (auto-generated or custom)
3. **Add the user to the group "Admins"**
    - During user creation → Add to group → Select `Admins`
    - Complete creation → Note login URL for future access

Consider Questions & Suggested Answers:

1. **Why are groups used with users?**
    - To manage permissions for multiple users collectively; easier than assigning policies individually.

2. **When should you add policies to a user directly?**
    - For unique permissions that apply only to that specific user.
3. **When should you add policies to a group?**
    - When multiple users need the same set of permissions (common practice).
4. **If you delete a group, what happens to the users assigned to that group?**
    - Users remain but lose the permissions granted via that group.

Exercise 3: Setting Up the AWS CLI

Concepts:

- AWS CLI allows automating tasks, reduces human errors, and is ideal for scripting.
- Access Key & Secret Access Key are required for CLI authentication.
- Security best practice: rotate keys regularly.

Tasks & Step-by-Step Solutions:

1. **Create Access Key for IAM user**
    - IAM → Users → Select your user → Security credentials → Create access key → Note Access Key ID & Secret Access Key (download CSV for safety).
2. **Install AWS CLI v2**
    - Follow installation for your OS from AWS CLI v2
3. **Configure AWS CLI**

Open terminal → Run:

```
aws configure
```

    -
    - Enter:
        - Access Key ID
        - Secret Access Key
        - Default region (e.g., us-east-1)
        - Default output format (json or table)
4. **Test CLI commands**

Example: List EC2 instances:

```
aws ec2 describe-instances
```

    -
    - Explore other commands via AWS CLI command reference

Consider Questions & Suggested Answers:

1. **How often should you rotate your AWS access key?**
    - Every 90 days (or as per company policy) for security best practices.
2. **When would you use AWS CLI versus Management Console?**
    - CLI: automation, scripting, batch operations, reproducibility.
    - Console: exploration, visual management, learning, and quick ad-hoc tasks.
3. **How can you retrieve a secret access key?**
    - Secret Access Key is only visible once at creation.
    - If lost, you must generate a new key pair.

Exercise 4: Setting Up a Budget

Concepts:

- AWS Budgets helps prevent unexpected charges by alerting you when spending approaches limits.
- **Actual cost** = money already spent.
- **Forecasted cost** = predicted spending based on usage trends.
- Budgets can be segmented by service, linked account, region, or tag.

Tasks & Step-by-Step Solutions:

1. **Create a budget in AWS Budgets**
    - AWS Console → Budgets → Create budget → Select **Cost Budget**
2. **Set budget details**
    - Period: Monthly
    - Amount: Your comfortable limit (e.g., $5)
    - Threshold: Forecasted cost = 100%
3. **Set notification**
    - Add your email → Receive alerts when threshold is exceeded

Consider Questions & Suggested Answers:

1. **Difference between actual cost threshold and forecasted cost threshold:**
    - **Actual cost threshold**: triggers when you have already spent the set amount.
    - **Forecasted cost threshold**: triggers when projected spending is expected to reach the set amount.
2. **Vectors to segment budgets:**

- Service (e.g., EC2, S3)
- Linked AWS accounts
- Region
- Tags (project, department, environment)

Exercise 5: Creating an EC2 Instance

Concepts:

- EC2 = virtual server in the cloud to run code and applications.
- Instance type = defines CPU, memory, storage, and network capacity.
- Key pair = used for secure SSH (Linux) or RDP (Windows) access.
- AMI (Amazon Machine Image) = template for your instance (OS + software).

Tasks & Step-by-Step Solutions:

1. **Create an EC2 instance**
    - AWS Console → EC2 → Launch Instances →
        - **Name:** Your choice
        - **AMI:** Amazon Linux 2
        - **Instance type:** t3.micro (free tier eligible)
        - Default options for storage, network, etc.
2. **Create a key pair**
    - During instance launch → Create new key pair → Name it → Download `.pem` file → Keep it safe (needed for SSH access)
3. **Launch instance**
    - Review → Launch → Wait until instance state = Running

Consider Questions & Suggested Answers:

1. **Tasks best suited for each instance type category:**
    - **t (General Purpose):** Web servers, dev/test environments
    - **m (Compute Optimized):** CPU-heavy apps like batch processing

- **r (Memory Optimized):** Databases, caching, in-memory analytics
- **g/p (GPU/Accelerated):** Machine learning, graphics rendering

2. **Instance type for your product domain:**
    - Depends on workload: e.g., web apps → t3, ML → g4, database → r5
3. **Other categories of AMIs:**
    - Windows Server, Ubuntu, Red Hat, SUSE Linux
    - Prebuilt software stacks in AWS Marketplace (e.g., WordPress, Docker, Node.js)

Exercise 6: EC2 Security Groups

Concepts:

- **Security Groups (SGs)** = virtual firewalls for EC2 instances controlling inbound/outbound traffic.
- SGs protect instances from unwanted access while allowing required traffic.
- Each EC2 instance can have multiple SGs attached.
- VPC security groups operate at the network level within a Virtual Private Cloud.

Tasks & Step-by-Step Solutions:

1. **Create a new EC2 security group**
    - AWS Console → EC2 → Security Groups → Create Security Group →
        - Name: Your choice
        - VPC: Select the default or your VPC
2. **Add incoming rules**
    - **SSH (Port 22):** Source = Your IP address
    - **Port 3000:** Source = Anywhere (0.0.0.0/0)
3. **Attach security group to your EC2 instance**
    - EC2 → Instances → Select instance → Actions → Networking → Change Security Groups → Attach your new SG

Consider Questions & Suggested Answers:

1. **What does an EC2 security group protect the instance from?**
    - Blocks unwanted inbound/outbound traffic; allows only permitted IPs/ports.
2. **What is a VPC security group?**

- Security group applied at the VPC/network level; controls traffic for all instances in that VPC.

3. **How many security groups can be attached to an EC2 instance?**

    - Up to **5 security groups per instance** (default limit, can be increased).

Exercise 7: Prepping the EC2 Instance

Concepts:

- SSH provides secure remote access to EC2 instances.
- Key pairs authenticate access instead of passwords.
- Dependencies must be installed to run your applications.
- Security best practice: limit SSH access to known IPs.

Tasks & Step-by-Step Solutions:

1. **SSH into your EC2 instance**

Open terminal → Run:

```
ssh -i ~/.ssh/my_keypair.pem ec2-user@<public_ip_of_instance>
```

    -
    - Replace `my_keypair.pem` with your downloaded key and `<public_ip_of_instance>` with your instance IP.

Ensure key permissions are secure:

```
chmod 400 ~/.ssh/my_keypair.pem
```

    -

2. **Install dependencies for your web application**

Example for Node.js app:

```
sudo yum update -y
sudo yum install -y git
sudo yum install -y nodejs npm
```

    -

- Adjust installation commands based on your application stack (Python, Java, etc.).

Consider Questions & Suggested Answers:

1. **Security risks of leaving SSH port open:**
    - Exposes instance to brute-force attacks.
    - Unauthorized access if key or credentials are compromised.
    - Always restrict access to known IP addresses.
2. **Troubleshooting via SSH:**
    - Check logs (e.g., `/var/log/`)
    - Inspect running processes (`ps`, `top`)
    - Test service connectivity (e.g., `curl localhost:3000`)
    - Restart services if needed (`sudo systemctl restart <service>`)

Exercise 8: Deploying to an EC2 Instance

Concepts:

- Deploying apps to EC2 can be manual (copying files) or automated using deployment tools.
- **SCP** (Secure Copy) transfers files securely over SSH.
- SSH is used to start and manage applications on EC2.
- Deployment automation improves efficiency and reduces errors.

Tasks & Step-by-Step Solutions:

1. **Prepare your web application**
    - Ensure all files, dependencies, and configuration are ready locally.
2. **Copy files to EC2 using SCP**

Linux/macOS:

```
scp -i ~/.ssh/my_keypair.pem -r /local/app/path
ec2-user@<public_ip>:/home/ec2-user/
```

    -
    - **Windows:** Use WinSCP or PowerShell with SCP command (see link).

3. **SSH into your EC2 instance and start the app**

Connect via SSH:

```
ssh -i ~/.ssh/my_keypair.pem ec2-user@<public_ip>
```

    -

Navigate to app folder → Run app (example for Node.js):

```
cd /home/ec2-user/app
npm install
```

```
npm start
```

-

Consider Questions & Suggested Answers:

1. **How could you automate the deployment process?**
    - Use CI/CD pipelines with **AWS CodeDeploy**, **GitHub Actions**, **Jenkins**, or **AWS CodePipeline**.
    - Scripts can automate SCP transfer, dependency installation, and service start.
2. **Deployment tools supporting AWS:**
    - **AWS CodeDeploy / CodePipeline** (native AWS)
    - **Jenkins** (plugin support for EC2)
    - **GitHub Actions** (AWS CLI or SDK integration)
    - **Ansible / Terraform** for infrastructure + deployment automation

Exercise 9: More Deploying to EC2 Instances

Concepts:

- User Data scripts run automatically when a new EC2 instance launches.
- Automates deployment tasks like cloning apps, installing dependencies, and starting services.
- GitHub can be used as a source for application code.
- Security and maintainability are important when selecting deployment methods.

Tasks & Step-by-Step Solutions:

Write a shell script for deployment
Example `deploy.sh` for Linux EC2:

```
#!/bin/bash
yum update -y
yum install -y git nodejs npm
cd /home/ec2-user
git clone https://github.com/<username>/<repo>.git app
cd app
npm install
npm start &
```

1.
    - Make script executable: `chmod +x deploy.sh`
2. **Create a new EC2 instance**
    - During setup → In **Advanced Details** → Paste the script into **User Data**
3. **Launch the instance**
    - Instance runs the script automatically at first boot
4. **Verify deployment**
    - Open browser → `http://<public_ip>:<app_port>`

- Confirm the app is running

Consider Questions & Suggested Answers:

1. **What kind of commands would you add to User Data?**
    - Install dependencies
    - Configure environment variables
    - Start services or applications
    - Create directories or set permissions
2. **Does User Data solve all deployment needs? What's missing?**
    - User Data only runs at first boot → no updates after launch
    - Lacks version control, rollback, and automated redeployments
    - For ongoing deployment, use CI/CD pipelines (CodeDeploy, CodePipeline, Jenkins, GitHub Actions)

Exercise 10: Creating an Elastic IP

Concepts:

- **Public IPs** allow internet access to EC2 instances.
- **Elastic IP (EIP)** is a static public IPv4 address that remains the same until released.
- Useful for apps that need a fixed IP even if the instance restarts or changes.
- AWS charges for unused Elastic IPs to prevent waste.

Tasks & Step-by-Step Solutions:

1. **Create a new Elastic IP**
    - AWS Console → EC2 → Elastic IPs → Allocate Elastic IP address
    - Scope: VPC → Allocate
2. **Associate Elastic IP with EC2 instance**
    - Select the Elastic IP → Actions → Associate Elastic IP
    - Choose your EC2 instance → Select private IP → Associate
3. **Verify application access**
    - Ensure your app is running on EC2 (e.g., port 3000)

Open browser →

```
http://<elastic_ip>:3000
```

-
- If it times out, verify security group allows inbound traffic on port 3000

Consider Questions & Suggested Answers:

1. **Difference between VPC Elastic IP and EC2 Elastic IP:**

- **VPC Elastic IP:** Used in modern VPC-based networking; can be remapped between instances.
- **EC2-Classic Elastic IP:** Older model (deprecated); tied to EC2-Classic networking.

2. **Cost of an Elastic IP:**
    - Free when **attached to a running EC2 instance**.
    - Charged when **unassociated** or attached to a stopped instance.
    - Additional charges for multiple Elastic IPs per instance.

Exercise 12: Securing Your EC2 Instance

Concepts:

- **Principle of Least Privilege**: allow only the minimum access required.
- Best practice: EC2 instances should not be publicly accessible; traffic should flow through a **load balancer**.
- Security groups restrict network access but do not secure the application itself.

Tasks & Step-by-Step Solutions:

1. **Update EC2 security group ingress rules**
    - AWS Console → EC2 → Security Groups → Select instance SG
2. **Allow traffic only from the load balancer**
    - Add inbound rule:
        - Type: Custom TCP
        - Port: 3000
        - Source: **Load Balancer Security Group**
3. **Remove direct access rules**
    - Delete inbound rule for SSH (port 22)
    - Delete any rule allowing port 3000 from `0.0.0.0/0`
4. **Verify access via load balancer**
    - Open browser → Use **Load Balancer DNS name**
    - Confirm application is accessible only through the load balancer

Consider Questions & Suggested Answers:

1. **How can an EC2 instance still be insecure with restrictive security groups?**
    - Vulnerable application code (SQL injection, XSS)
    - Outdated OS packages or misconfigured services
    - Weak IAM roles or exposed credentials
    - Security groups do not protect against application-layer attacks
2. **How many OWASP Top 10 risks can EC2 security groups mitigate?**
    - Very few directly
    - Security groups mainly mitigate **network-level risks**
    - Most OWASP Top 10 risks are **application-layer issues** and require secure coding, WAFs, and proper authentication

Exercise 13: Creating an Amazon Machine Image (AMI)

Concepts:

- **AMI (Amazon Machine Image)** captures the full state of an EC2 instance (OS, software, configuration).
- AMIs are used for backup, cloning, scaling, and sharing environments.
- Backed by **EBS snapshots**; free tier covers limited snapshot storage.
- Cleaning up resources prevents unexpected charges.

Tasks & Step-by-Step Solutions:

1. **Create an AMI from your EC2 instance**
    - AWS Console → EC2 → Instances → Select instance
    - Actions → Image and templates → Create image
    - Name the AMI → Create
    - Wait until AMI status = Available
2. **Delete Load Balancer resources**
    - EC2 → Load Balancers → Delete load balancer
    - EC2 → Target Groups → Delete target group
3. **Terminate EC2 instance**
    - EC2 → Instances → Select instance → Terminate
4. **Delete EC2 security group**
    - EC2 → Security Groups → Delete the custom security group
5. **Release Elastic IP**
    - EC2 → Elastic IPs → Select → Release Elastic IP

Consider Questions & Suggested Answers:

1. **How can AMIs be used to share code or applications?**
    - Share AMIs across AWS accounts or regions.
    - Launch identical, preconfigured environments quickly.
    - Use AMIs as golden images for production consistency.
2. **Centralized AMI management in organizations:**
    - Many companies maintain a **platform or DevOps team** to manage hardened, approved AMIs.
    - Ensures security, compliance, and standardized environments.

Exercise 20: Creating an S3 Bucket

Concepts:

- **Amazon S3** is object storage used by almost all cloud applications.
- Buckets store objects (files) and scale automatically.
- Public S3 buckets are a major security risk if misconfigured.

Tasks & Step-by-Step Solutions:

1. **Create an S3 bucket**
    - AWS Console → S3 → Create bucket
    - Choose a globally unique bucket name
    - Select a region
    - **Uncheck "Block all public access"** (for learning only)
    - Leave all other options as default → Create bucket
2. **Upload files to the bucket**
    - Open the bucket → Upload → Add files → Upload

Consider Questions & Suggested Answers:

1. **Why shouldn't S3 buckets be public?**
    - Risk of data leaks and breaches
    - Public access can expose sensitive or regulated data
    - Misconfigured public buckets are a common security incident
2. **What file types can be stored in S3?**
    - Any file type: images, videos, logs, backups, binaries, datasets, static websites

Exercise 21: Accessing S3 Objects

Concepts:

- S3 access is controlled at **bucket** and **object** levels.
- **ACLs (Access Control Lists)** define who can read/write an object.
- Public objects are a major data-leak risk if misused.
- Prefer controlled access mechanisms over public ACLs.

Tasks & Step-by-Step Solutions:

1. **Select an S3 object**
    - AWS Console → S3 → Open your bucket → Click an uploaded object
2. **Edit the object ACL**
    - Open **Permissions** tab
    - In **Access control list (ACL)** → Click **Edit**
3. **Make the object public**
    - Check **Everyone (public access)** under **Objects – Read**
    - Acknowledge the warning → Save changes
4. **Verify public access**
    - Click **Object URL**
    - Open link in browser → File should be accessible

Consider Questions & Suggested Answers:

1. **When should S3 objects be public?**
    - Static website assets (images, CSS, JS)

- Public datasets or open resources
- Never for sensitive, private, or regulated data

2. **Other ways to provide public access without making objects public:**
    - **Pre-signed URLs** (temporary, time-limited access)
    - **CloudFront** with signed URLs or signed cookies
    - API-based access through a backend service

Exercise 22: S3 Lifecycle Rules

Concepts:

- **S3 Lifecycle rules** automate object management over time.
- Used to **expire**, **transition**, or **archive** objects.
- Helps reduce storage costs and operational overhead.

Tasks & Step-by-Step Solutions:

1. **Open bucket lifecycle settings**
    - AWS Console → S3 → Select your bucket
    - Go to **Management** tab
2. **Create a lifecycle rule**
    - Click **Create lifecycle rule**
    - Name the rule
    - Scope: **Apply to all objects**
3. **Configure rule action**
    - Select **Expire current versions of objects**
    - Set expiration period (e.g., 30 days)
4. **Save the rule**
    - Review → Click **Create rule**

Consider Questions & Suggested Answers:

1. **Objects suitable for lifecycle rules:**
    - Logs and audit files

- Temporary uploads
- Backups and snapshots
- Archived or infrequently accessed data

2. **Most commonly used lifecycle rules:**
    - Transition to **S3 Glacier** for long-term storage
    - Expire objects after a fixed retention period

Exercise 23: S3 Storage Classes

Concepts:

- S3 offers multiple **storage classes** to balance cost, access speed, and durability.
- **Standard** = frequent access, higher cost.
- **Standard-IA** = infrequent access, lower cost, retrieval fee applies.
- **Glacier** = long-term archival, very low cost, slow retrieval.
- **Intelligent-Tiering** automatically optimizes cost based on access patterns.

Tasks & Step-by-Step Solutions:

1. **Select an object in S3**
    - AWS Console → S3 → Open your bucket → Click an object
2. **Change the storage class**
    - In **Storage class** section → Click **Edit**
    - Change from **Standard** → **Standard-IA**
    - Click **Save changes**

Consider Questions & Suggested Answers:

1. **Files best suited for S3 Glacier:**
    - Compliance and audit logs
    - Long-term backups
    - Historical data rarely accessed
    - Disaster recovery archives
2. **How S3 Intelligent-Tiering works:**

- Automatically moves objects between access tiers
- No performance impact
- Optimizes cost without manual intervention
- Best for unpredictable access patterns

Exercise 24: S3 Website Hosting

Concepts:

- **S3 static website hosting** serves files directly from S3 over HTTP.
- Ideal for static content: HTML, CSS, JavaScript, images.
- Simple, scalable, and cost-effective—no servers required.

Tasks & Step-by-Step Solutions:

1. **Open bucket properties**
    - AWS Console → S3 → Select your bucket
    - Go to **Properties** tab
2. **Enable static website hosting**
    - Scroll to **Static website hosting** → Click **Edit**
    - Select **Enable**
3. **Configure site documents**
    - **Index document:** e.g., `index.html`
    - **Error document:** e.g., `error.html`
4. **Save and test**
    - Click **Save changes**
    - Use the **Endpoint URL** shown in the Static website hosting panel
    - Open it in a browser to verify

Consider Questions & Suggested Answers:

1. **Objects best served by an S3 static site:**

- Static websites
- Single-page applications (React, Vue, Angular builds)
- Images, CSS, JavaScript, fonts

2. **Difference between S3 static website vs public S3 bucket:**
    - **Static website:** Uses a website endpoint, supports index/error pages, HTTP-only.
    - **Public bucket:** Direct object access; no routing, no index/error handling.

Exercise 25: S3 Bucket Policies

Concepts:

- **Bucket policies** apply permissions to all objects in a bucket at once.
- Prefer bucket policies over per-object ACLs for consistency and safety.
- Read-only public buckets are common for static website assets.
- Policies use **IAM policy JSON syntax**.

Tasks & Step-by-Step Solutions:

1. **Get a read-only public bucket policy**
    - Use the standard read-only policy template (Allow `s3:GetObject`)

Example policy (replace `YOUR_BUCKET_NAME`):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/*"
    }
  ]
}
```

2.
3. **Apply the bucket policy**
    - AWS Console → S3 → Select bucket → **Permissions** tab
    - **Bucket policy** → Click **Edit**
    - Paste the policy → Replace bucket name → **Save changes**

4. **Verify public access**
    - Open object URLs in a browser
    - All objects should now be publicly readable

Consider Questions & Suggested Answers:

1. **Bucket policy vs object ACL:**
    - **Bucket policy:** Centralized, applies to all objects, easier to manage
    - **Object ACL:** Per-object permissions, harder to maintain at scale
2. **Other uses of bucket policies:**
    - Restrict access by IP address
    - Allow access only from a VPC or CloudFront
    - Grant access to specific IAM roles or AWS accounts
    - Enforce encryption requirements

Exercise 26: Creating a DynamoDB Table

Concepts:

- **Amazon DynamoDB** is a fully managed, serverless **NoSQL key-value and document database**.
- Automatically scales and provides single-digit millisecond latency.
- Data is stored as **items** with flexible attributes (schema-less beyond keys).

Tasks & Step-by-Step Solutions:

1. **Create a DynamoDB table**
    - AWS Console → DynamoDB → Tables → **Create table**
    - **Table name:** Your choice
    - **Partition key:** `id` (Type: String)
    - Leave all other options as **Default settings**
    - Click **Create table**
2. **Create items in the table**
    - Open the table → Go to **Items** tab
    - Click **Create item**
    - Add attributes (e.g., `name`, `email`, `createdAt`, `status`)
    - Save multiple items for later use

Consider Questions & Suggested Answers:

1. **Difference between DynamoDB tables and S3 buckets:**
    - **DynamoDB:** Structured access via keys; low-latency reads/writes; used for application data.

- **S3:** Object storage for files; not optimized for querying individual records.

2. **How DynamoDB differs from traditional databases:**
    - No fixed schema (beyond keys)
    - No joins or complex queries
    - Fully managed, no servers to maintain
    - Scales automatically without manual sharding

Exercise 28: DynamoDB Indexes

Concepts:

- **Indexes** in DynamoDB improve query performance on attributes other than the primary key.
- **Global Secondary Index (GSI):** Can use any attribute as the partition (and optional sort) key, across the whole table.
- Proper index design is crucial for efficient, low-latency queries.

Tasks & Step-by-Step Solutions:

1. **Open your DynamoDB table**
    - AWS Console → DynamoDB → Select your table → **Indexes** tab
2. **Create a Global Secondary Index**
    - Click **Create index**
    - **Partition key:** Choose one of the non-primary key attributes from your items (e.g., `email`)
    - **Index name:** Give a descriptive name (e.g., `EmailIndex`)
    - Click **Create index**
3. **Verify the index**
    - Index will appear in the **Indexes** tab
    - Can now query the table using the indexed attribute without scanning the full table

Consider Questions & Suggested Answers:

1. **When to use a Global Secondary Index?**
    - When you need to query the table on non-primary key attributes

- For additional query patterns without scanning the entire table
- For sorting or filtering results efficiently

2. **Difference between Local and Global Secondary Index:**
    - **Local Secondary Index (LSI):** Uses the same partition key as the table, allows a different sort key
    - **Global Secondary Index (GSI):** Can use a completely different partition and sort key from the table

Exercise 32: Creating an RDS Database

Concepts:

- **Amazon RDS** is a fully managed relational database service.
- AWS handles **patching, backups, replication, and monitoring**.
- Choosing the right **engine type** (MySQL, PostgreSQL, etc.) and **instance size** affects performance, cost, and compatibility.
- Free tier available for light workloads and learning purposes.

Tasks & Step-by-Step Solutions:

1. **Open RDS dashboard**
    - AWS Console → RDS → Databases → Create database
2. **Select database engine**
    - Example: MySQL, PostgreSQL, MariaDB, etc.
3. **Create the database instance**
    - Choose **Easy create**
    - Select **Free tier**
    - Review defaults (storage, instance class, username/password)
    - Click **Create database**
4. **Verify instance**
    - Wait until **DB status = Available**
    - Note **endpoint** for connecting your application

Consider Questions & Suggested Answers:

1. **Database maintenance handled by AWS RDS:**

- Automatic OS and database patching
- Backups and snapshots
- Multi-AZ replication for high availability
- Monitoring and alerts

2. **What to consider when selecting an RDS instance type:**
    - Workload type (development, production, transactional, analytical)
    - CPU, memory, and storage requirements
    - High availability needs (Multi-AZ)
    - Cost and free tier eligibility

Exercise 33: RDS Accessibility

Concepts:

- RDS instances can be made **publicly accessible** to connect from outside AWS (e.g., your local computer).
- **Security groups** control which IPs and ports can access the database.
- Public accessibility is **not recommended for production**—use VPN, VPC endpoints, or bastion hosts instead.

Tasks & Step-by-Step Solutions:

1. **Edit RDS DB instance for public access**
    - AWS Console → RDS → Databases → Select your DB instance
    - **Modify DB** → Set **Publicly Accessible = True** → Save
2. **Update the security group**
    - Go to **EC2** → **Security Groups** associated with your RDS instance
    - Add an **Ingress rule**:
        - Protocol/Port = database engine (e.g., 3306 for MySQL)
        - Source = **My IP** → Save
3. **Connect to RDS from local client**
    - Use **endpoint + port + username/password** in a DB client (e.g., MySQL Workbench, pgAdmin)

Consider Questions & Suggested Answers:

1. **How to secure an RDS DB instance:**
    - Use private subnets instead of public access
    - Restrict security group IP ranges

- Enable encryption at rest and in transit
- Use IAM authentication where possible
- Enable automated backups and Multi-AZ deployment

2. **What “Publicly Accessible” does:**
    - Allows the DB endpoint to be reachable from outside the VPC
    - Security groups still control which IPs can connect

Exercise 36: AWS Regions

Concepts:

- **AWS Regions** are separate geographic areas hosting AWS data centers.
- **Availability Zones (AZs)** are isolated locations within a region.
- Some AWS services are **regional** (like EC2, RDS, S3 buckets), while others are **global** (like IAM, CloudFront).
- Choosing the correct region affects **latency, compliance, cost, and availability**.

Tasks & Step-by-Step Solutions:

1. **Switch AWS Regions in the Console**
    - Top-right corner of AWS Console → **Select a region** from the dropdown
2. **Check region restrictions**
    - Some regions may not support all services or require special account approval
3. **Identify global vs regional services**
    - Global services: IAM, Route 53, CloudFront
    - Regional services: EC2, S3, RDS

Consider Questions & Suggested Answers:

1. **Differences between regions besides location:**
    - Service availability and limits
    - Pricing variations
    - Latency to end-users
    - Compliance with local regulations
2. **Region closest to you or your users:**

- For Pakistan: **Mumbai (ap-south-1)** or **Singapore (ap-southeast-1)**
- Pick regions close to your users for lower latency

Exercise 37: AWS Availability Zones

Concepts:

- **Availability Zones (AZs)** are isolated data centers within an AWS region.
- AZs are **physically separate but connected via low-latency networks**.
- Using multiple AZs improves **high availability, fault tolerance, and disaster recovery**.

Tasks & Step-by-Step Solutions:

1. **Modify deployment AZ for existing resources**
    - **EC2:** Launch instance → **Availability Zone** → select a different AZ
    - **RDS:** Modify DB instance → **Availability & durability** → choose AZ (or Multi-AZ) → Apply changes
2. **Verify AZ changes**
    - Check the **instance details** in the console to confirm the selected AZ

Consider Questions & Suggested Answers:

1. **How are AZs connected?**
    - High-speed, low-latency private links
    - Separate power, cooling, and physical infrastructure for isolation
2. **Why select multiple AZs?**
    - Ensures **service continuity** if one AZ fails
    - Supports **Multi-AZ deployments** for databases and critical services
    - Improves **fault tolerance and disaster recovery**

Exercise 39: Creating a VPC

Concepts:

- **VPC (Virtual Private Cloud)** is a logically isolated network in AWS.
- Provides control over **IP address ranges, subnets, routing, and security**.
- All AWS resources that need networking—EC2, RDS, Lambda with VPC access—use VPCs.
- VPCs improve **security** by isolating resources and controlling inbound/outbound traffic.

Tasks & Step-by-Step Solutions:

1. **Create a new VPC**
    - AWS Console → VPC → **Create VPC**
    - **Name:** Choose a descriptive name (e.g., `MyVPC`)
    - **CIDR block:** `10.0.0.0/16`
    - Leave other options default → Click **Create VPC**
2. **Verify the VPC**
    - Go to **Your VPCs** → Confirm the VPC is created with the correct CIDR

Consider Questions & Suggested Answers:

1. **How does VPC protect AWS resources?**
    - Controls inbound/outbound traffic with **security groups** and **network ACLs**
    - Isolates resources from other users
    - Allows private subnets not accessible from the Internet
2. **Why other AWS services use VPCs besides EC2:**

- Services like RDS, ElastiCache, Redshift, and Lambda need **networking, IP addressing, and security isolation**

- Centralizes control of all networked resources for security and connectivity

Exercise 41: Creating an Internet Gateway

Concepts:

- An **Internet Gateway (IGW)** is a **horizontally scaled, redundant AWS component** that allows communication between your VPC and the Internet.
- Required for **public subnets** to access the Internet or be accessed from the Internet.
- Does **not provide NAT**; resources must have public IPs to use it.

Tasks & Step-by-Step Solutions:

1. **Create a new Internet Gateway**
    - AWS Console → VPC → Internet Gateways → **Create Internet Gateway**
    - Name it (e.g., `MyIGW`) → Click **Create**
2. **Attach the Internet Gateway to your VPC**
    - Select the created IGW → **Actions** → **Attach to VPC**
    - Choose your VPC → Click **Attach**
3. **Verify attachment**
    - In the IGW details → Check **VPC attachment status = Attached**

Consider Questions & Suggested Answers:

1. **What is an Internet Gateway?**
    - A VPC component that allows traffic between your VPC and the Internet
    - Provides a **target for routing tables** in public subnets
2. **Why are IGWs used to give Internet access?**
    - Acts as a **bridge** between private VPC networks and the Internet
    - Enables EC2 instances with public IPs or NAT to communicate externally

Exercise 42: Creating a Route Table

Concepts:

- **Route tables** define how network traffic flows in a VPC.
- Each VPC has a **main route table** for routing traffic by default.
- To allow Internet access from a **public subnet**, a route for `0.0.0.0/0` pointing to the **Internet Gateway** is required.
- Route tables can also control traffic to private subnets, VPNs, and peered VPCs.

Tasks & Step-by-Step Solutions:

1. **Create a new route table**
    - AWS Console → VPC → Route Tables → **Create route table**
    - Name it (e.g., `PublicRouteTable`) → Associate with your VPC → Click **Create**
2. **Add a route to the Internet Gateway**
    - Select your route table → **Routes** → **Edit routes** → **Add route**
    - **Destination:** `0.0.0.0/0` (all traffic)
    - **Target:** Select your **Internet Gateway** → Save changes
3. **Set as main route table for the VPC**
    - Go to **Route Table** → **Actions** → **Set as main route table**

Consider Questions & Suggested Answers:

1. **How do route tables increase VPC security?**
    - Control which traffic is allowed to flow in/out of subnets
    - Prevents unauthorized Internet access to private subnets

2. **Other routes you may add:**
    - Routes to **VPN connections**
    - Routes to **VPC peering connections**
    - Routes to **NAT Gateways** for private subnet Internet access

Exercise 43: Public & Private Subnets

Concepts:

- **Public subnets:** Can send/receive traffic directly to/from the Internet via an Internet Gateway.
- **Private subnets:** Cannot be accessed directly from the Internet; use a **NAT Gateway** for outbound Internet access.
- This setup improves **security** by keeping sensitive resources (databases, backend services) in private subnets.

Tasks & Step-by-Step Solutions:

1. **Create a NAT Gateway**
    - AWS Console → VPC → NAT Gateways → **Create NAT Gateway**
    - **Subnet:** Choose a **public subnet**
    - **Elastic IP:** Allocate a new one or select an existing EIP → Create
2. **Create & configure a route table for private subnet**
    - AWS Console → VPC → Route Tables → **Create route table**
    - **Name:** e.g., `PrivateRouteTable` → Associate with VPC → Create
    - Edit routes → Add route:
        - **Destination:** `0.0.0.0/0`
        - **Target:** NAT Gateway → Save
3. **Associate route table with private subnet**
    - Select **PrivateRouteTable → Subnet Associations → Edit → Select private subnet → Save**

Consider Questions & Suggested Answers:

1. **Why not provision all resources in public subnets?**
    - Exposes them directly to the Internet → security risk
    - Makes sensitive data more vulnerable to attacks
2. **Which resources go in which subnet?**
    - **Public subnet:** NAT Gateway, Load Balancers, Bastion Hosts, web servers accessible from Internet
    - **Private subnet:** Databases, application servers, internal services, backend APIs

Exercise 45: Amazon Route 53

Concepts:

- **Amazon Route 53** is AWS's **scalable Domain Name System (DNS) service**.
- DNS translates human-readable domain names (like `example.com`) into IP addresses so users can reach your applications.
- Route 53 provides **domain registration, DNS routing, health checks, and traffic management**.
- It supports advanced routing policies: **weighted, latency-based, failover, and geolocation routing**.

Tasks & Step-by-Step Solution:

1. **Understand DNS and Route 53**
    - Read the AWS docs and DNS basics
    - Key points:
        - Domains → Hosted Zones
        - Records → Map domain to resources (EC2, S3, Load Balancers)
        - Health checks → Ensure traffic is routed to healthy endpoints

Consider Questions & Suggested Answers:

1. **Why is Route 53 considered a core AWS service?**
    - Every web application relies on DNS for accessibility
    - Route 53 integrates tightly with other AWS services (ELB, S3, CloudFront)
    - Provides global, reliable, and highly available DNS service
2. **Use cases for Route 53:**
    - Map domain names to EC2 instances, S3 static websites, or load balancers

- Route traffic based on **latency, location, or failover needs**
- Monitor endpoint health and automatically reroute traffic if a resource fails

Exercise 52: IAM Policies

Concepts:

- **IAM Policies** define **who can do what on which resources** in AWS.
- Two types:
    - **Managed Policies:** Pre-built or customer-managed, reusable across users/groups/roles.
    - **Inline Policies:** Attached directly to a single user, group, or role.
- Policies can be created using a **Visual Editor** or by writing **JSON**.
- Key elements of a policy:
    - **Effect:** Allow or Deny
    - **Action:** AWS service actions (e.g., `s3:PutObject`)
    - **Resource:** The specific resource(s) the action applies to
    - **Principal:** Who the policy applies to (user, role, service)

Tasks & Step-by-Step Solution:

1. **Create a policy using the Visual Editor**
    - AWS Console → IAM → Policies → **Create Policy** → Visual Editor
    - Select a service (e.g., S3) → Add actions (e.g., `s3:ListBucket`, `s3:GetObject`)
    - Specify resources (e.g., your S3 bucket) → Review → Name & Create
2. **Create a policy using JSON**
    - AWS Console → IAM → Policies → **Create Policy** → JSON tab

Example JSON:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject"],
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

-
- Review → Name & Create

Consider Questions & Suggested Answers:

1. **What types of values does each property on a policy expect?**
    - **Effect:** `"Allow"` or `"Deny"` (string)
    - **Action:** Array of strings (service:action)
    - **Resource:** ARN string(s)
    - **Principal:** User, role, AWS service, or account
2. **Different principals allowed in a policy:**
    - IAM users
    - IAM roles
    - AWS services (e.g., `ec2.amazonaws.com`)
    - AWS accounts (account ID)

Exercise 53: IAM Roles

Concepts:

- **IAM Roles** are identities that can be assumed by **AWS resources** or users to gain temporary permissions.
- Roles are used instead of embedding credentials in applications or EC2 instances.
- **Instance Profiles** are the bridge that allows an EC2 instance to assume a role.
- Common use: granting EC2, Lambda, or other services permission to access AWS resources securely.

Tasks & Step-by-Step Solution:

1. **Create a new IAM role**
    - AWS Console → IAM → Roles → **Create Role**
    - Select **AWS service** → **EC2**
    - Attach policies:
        - Example: `AmazonS3ReadOnlyAccess`, `CloudWatchAgentServerPolicy`, plus your custom policy
    - Name the role → Create Role
2. **Attach the role to an EC2 instance**
    - Launch an EC2 instance using your saved AMI
    - Under **Configure Instance** → **IAM Role**, select the role you created
    - Complete launch
3. **Verify role permissions**
    - SSH into EC2 → run commands to access resources (e.g., `aws s3 ls`)
    - No keys needed; permissions are granted via the role

Consider Questions & Suggested Answers:

1. **Differences between users and roles:**
    - **Users:** Permanent credentials, tied to a person or service account
    - **Roles:** Temporary permissions, assumed by resources or users, no long-term credentials
2. **Why EC2 instances need an instance profile:**
    - The instance profile **associates a role with an EC2 instance**
    - Allows the EC2 instance to automatically inherit permissions from the role

Exercise 57: Creating a Lambda Function

Concepts:

- **AWS Lambda** is a **serverless compute service** that runs your code without provisioning servers.
- Code executes in response to **events** (e.g., HTTP requests, S3 uploads, DynamoDB updates).
- Lambda automatically **scales** by running multiple function instances in parallel based on incoming requests.
- Ideal for short-lived tasks, microservices, and event-driven architectures.

Tasks & Step-by-Step Solution:

1. **Create a Lambda function**
    - AWS Console → Lambda → **Create function**
    - Choose **Author from scratch**
    - Set **Function name**
    - Runtime: Choose a familiar language (e.g., Node.js, Python)
    - Use default **execution role** (or a custom IAM role if needed) → Create
2. **Add simple code (optional)**
    - Default Node.js example returns a basic JSON response

Example:

```
exports.handler = async (event) => {
    return {
        statusCode: 200,
        body: JSON.stringify('Hello from Lambda!')
    };
};
```

    -
3. **Test the Lambda function**
    - AWS Console → Lambda → **Test**
    - Create a test event (use default template) → Execute
    - Confirm the function returns the expected output

Consider Questions & Suggested Answers:

1. **When to use Lambda instead of EC2:**
    - Short-lived or event-driven tasks
    - Avoid managing servers
    - Automatic scaling is needed
2. **How scaling works with Lambda:**
    - AWS runs multiple instances of the function **in parallel** based on incoming requests
    - No manual server provisioning required
    - Scaling is **instantaneous and automatic** within AWS concurrency limits

Exercise 59: Lambda Triggers

Concepts:

- **Lambda Triggers** allow AWS Lambda functions to run in response to events from other AWS services.
- Example: HTTP requests via **API Gateway** trigger a Lambda function, which then processes the request and returns a response.
- Lambda supports many triggers: **S3 uploads, DynamoDB streams, CloudWatch events, SNS messages**, etc.
- Event-driven architecture allows **serverless, scalable, and loosely coupled applications**.

Tasks & Step-by-Step Solution:

1. **Create a REST API in API Gateway**
    - AWS Console → API Gateway → **Create API** → **REST API**
    - Create a **resource** (e.g., `/test`) and **method** (e.g., `GET`)
2. **Integrate API Gateway with Lambda**
    - In the method execution, select **Lambda Function** as the integration type
    - Specify the Lambda function you created earlier
    - Give API Gateway permission to invoke Lambda (it usually auto-adds)
3. **Modify Lambda code for response**

Example Node.js code for JSON response:

```
exports.handler = async (event) => {
    return {
        statusCode: 200,
        body: JSON.stringify({ message: "Hello from API Gateway + Lambda!" })
```

```
    };
};
```

    -

4. **Test the trigger**
    - Deploy the API → Get the endpoint URL
    - Access it via browser or `curl` → Confirm it triggers Lambda and returns correct response

Consider Questions & Suggested Answers:

1. **Other useful triggers for Lambda:**
    - **S3 object upload** → process files automatically
    - **DynamoDB Streams** → handle database changes
    - **CloudWatch Events / EventBridge** → scheduled tasks
    - **SNS/SQS messages** → event-driven workflows
2. **Why AWS adopted an event-based model:**
    - Scales automatically per event
    - Reduces idle compute cost (pay only for execution time)
    - Enables decoupled, reactive applications

Exercise 61: AWS CloudFormation

Concepts:

- **AWS CloudFormation** is AWS's **Infrastructure as Code (IaC) service**.
- Allows you to **provision, update, and manage AWS resources** using **templates** in JSON or YAML.
- Templates define resources, their properties, and dependencies in a **declarative way**.
- **Stacks** are collections of AWS resources created and managed together from a template.
- Using CloudFormation ensures **repeatability, consistency, and version control** for infrastructure.

Tasks & Step-by-Step Solution:

1. **Create a CloudFormation Stack**
    - AWS Console → CloudFormation → **Create stack** → **With sample template**
    - Choose a sample template (e.g., EC2 + S3)
    - Give the stack a name → **Next** → **Create stack**
2. **Monitor Stack Creation**
    - CloudFormation provisions all resources defined in the template
    - View progress in the **Events** tab
    - Once status is **CREATE_COMPLETE**, all resources are ready

Consider Questions & Suggested Answers:

1. **Why use CloudFormation instead of manual configuration?**
    - Automates provisioning → reduces human error
    - Infrastructure is **version-controlled and reproducible**

- Easier to scale, update, or replicate environments

2. **How does CloudFormation keep resources grouped?**
    - All resources defined in a template are **part of the same stack**
    - Stack lifecycle (create, update, delete) affects all resources together
    - Dependencies between resources are automatically managed

Exercise 68: CloudFront Distributions

Concepts:

- **Amazon CloudFront** is AWS's **Content Delivery Network (CDN)**.
- It delivers content (e.g., static files, images, videos) to users with **low latency** by caching at **edge locations** worldwide.
- A **distribution** defines how CloudFront serves content from an **origin** (S3, EC2, or custom server).
- Improves performance, reduces load on the origin, and enhances security via HTTPS and access controls.

Tasks & Step-by-Step Solution:

1. **Create a CloudFront distribution**
    - AWS Console → CloudFront → **Create Distribution** → **Web**
    - Set **Origin Domain Name** → Select your S3 bucket
    - Keep default settings (or enable caching, HTTPS if needed) → **Create Distribution**
2. **Wait for deployment**
    - Distribution can take several minutes to deploy
    - Status will change to **Deployed**
3. **Access files via CloudFront**
    - Copy the **CloudFront URL**
    - Append the object key (file name) → Open in browser
    - Verify the file loads via CloudFront, not directly from S3

Consider Questions & Suggested Answers:

1. **What edge locations to use:**
    - Choose locations **closest to your users** for lowest latency
    - CloudFront automatically serves from the nearest edge by default
2. **Types of origins for a distribution:**
    - **S3 buckets** (static content)
    - **EC2 instances** or **Elastic Load Balancers** (dynamic content)
    - **Custom origins** outside AWS (any HTTP server)

Exercise 71: Creating an Elastic Beanstalk Application

Concepts:

- **AWS Elastic Beanstalk** is a **Platform-as-a-Service (PaaS)** that simplifies deploying and managing applications.
- Groups resources like **EC2, load balancers, auto-scaling, and RDS** into an **application environment**.
- Supports multiple platforms: **Node.js, Java, Python, .NET, Docker**, etc.
- Separates **applications** (the code and configuration) from **environments** (staging, production) for testing and deployment flexibility.

Tasks & Step-by-Step Solution:

1. **Create an Elastic Beanstalk application**
    - AWS Console → Elastic Beanstalk → **Create Application**
    - Enter **Application Name**
2. **Choose platform and sample app**
    - Select your platform (e.g., Node.js or Java)
    - Choose the **sample application** option
3. **Create environment**
    - The wizard automatically creates an environment for the application
    - Wait until the environment status shows **Green / Ready**
4. **Access the application**
    - Use the provided URL to visit the deployed sample application
    - Verify it loads correctly

Consider Questions & Suggested Answers:

1. **Choosing a platform:**
    - Match the platform to your application's runtime (Node.js, Java, Python, etc.)
    - Consider dependencies, performance, and AWS support for the runtime
2. **Separating application and environment for testing:**
    - Use **multiple environments**: one for development, one for staging, one for production
    - Deploy new versions to staging first → test → promote to production
    - Supports **blue/green deployments** for safer rollouts

Exercise 76: Connecting to ElastiCache

Concepts:

- **Amazon ElastiCache** is a fully managed **in-memory caching service** (supports Redis and Memcached).
- Caching improves application speed by storing frequently accessed data in memory.
- Access to ElastiCache is **restricted to AWS resources** for security.
- Typically deployed in **private subnets** within a VPC for secure access.

Tasks & Step-by-Step Solution:

1. **Modify application code to use Redis**
    - Install a Redis client library for your runtime (Node.js: `ioredis` or `redis`, Python: `redis-py`)
    - Update code to connect to the **ElastiCache Redis cluster endpoint**

Example (Node.js):

```
 const Redis = require('ioredis');
const redis = new Redis({ host: '<elasticache-endpoint>', port: 6379
});

// Set a value
await redis.set('key', 'value');

// Get a value
const value = await redis.get('key');
console.log(value);
```

    -
2. **Modify ElastiCache security group**
    - Allow inbound traffic on **port 6379** from the **EC2 security group** used by your Elastic Beanstalk environment

3. **Deploy application to Elastic Beanstalk**
    - Push the updated code
    - Ensure the environment uses the modified security group
4. **Verify connection**
    - Test setting and retrieving cache values in your app
    - Monitor **connections and metrics** in the ElastiCache dashboard

Consider Questions & Suggested Answers:

1. **Why is ElastiCache access restricted outside AWS?**
    - For security: prevents unauthorized access to in-memory data
    - Ensures low latency by keeping traffic within AWS network
2. **Where to provision ElastiCache in a public/private subnet VPC?**
    - In a **private subnet**: not accessible from the public internet
    - Applications in the public subnet (like Elastic Beanstalk frontend) access it via VPC routing

Exercise 81: Consuming Kinesis Data

Concepts:

- **Amazon Kinesis Data Streams** allows real-time streaming of data for processing.
- A **consumer application** reads and processes data from the stream.
- Multiple consumers can read the same stream for parallel processing or different use cases.
- **Kinesis Client Library (KCL)** simplifies building consumers by handling shard coordination, checkpointing, and retry logic.

Tasks & Step-by-Step Solution:

1. **Set up a Kinesis Data Stream**
    - Create a Kinesis stream in AWS Management Console
    - Note the **stream name** and **region**
2. **Write a consumer application using KCL**
    - Install the KCL library for your language (Node.js, Java, Python)
    - Connect to the stream using your **AWS credentials**

Sample Node.js snippet:

```
const KCL = require('aws-kcl');

const recordProcessor = {
  initialize: (initializeInput, completeCallback) =>
completeCallback(),
  processRecords: (processRecordsInput, completeCallback) => {
    processRecordsInput.records.forEach(record => {
      const payload = Buffer.from(record.data, 'base64').toString();
      console.log('Received record:', payload);
    });
    completeCallback();
```

```
  },
  shutdown: (shutdownInput, completeCallback) => completeCallback()
};

KCL(recordProcessor).run();
```

    -

3. **Send data to the stream**
    - Use the AWS CLI or SDK to put sample records into the Kinesis stream
    - Verify your consumer logs the data
4. **Monitor and scale**
    - Multiple consumers can read from the same stream without interfering
    - Each consumer can process data independently or in parallel shards

Consider Questions & Suggested Answers:

1. **Why use multiple consumers on the same stream?**
    - Parallel processing: handle high-throughput data efficiently
    - Different consumers can perform different actions on the same data (e.g., analytics, monitoring, storage)
2. **What does Kinesis Client Library simplify?**
    - Shard management and coordination
    - Checkpointing and retry handling
    - Load balancing among consumers automatically

Exercise 91: Elastic Kubernetes Service (EKS)

Concepts:

- **Kubernetes** is an open-source container orchestration system that automates deployment, scaling, and management of containerized applications.
- **Amazon EKS** is AWS's managed Kubernetes service, handling control plane management, patching, and high availability.
- Kubernetes manages **Pods** (groups of containers), **Nodes** (VMs or EC2 instances), **Deployments**, and **Services** for scalable applications.
- Difference from Docker: Docker handles single containers, while Kubernetes orchestrates multiple containers across a cluster.

Tasks & Step-by-Step Solution:

1. **Learn Kubernetes basics**
    - Understand key concepts: Pod, Node, Cluster, Deployment, Service, Namespace
    - Watch tutorials or read guides on Kubernetes architecture
2. **Understand Amazon EKS**
    - EKS runs the Kubernetes control plane for you
    - You manage worker nodes (EC2 instances) or use Fargate for serverless compute
    - AWS handles patching, scaling, and high availability for the control plane
3. **Explore resources**
    - Check official AWS EKS documentation and Kubernetes guides
    - Look for tutorials showing creating a cluster, deploying a containerized app, and scaling pods

Consider Questions & Suggested Answers:

1. **How does Kubernetes differ from Docker?**
    - Docker: Runs single containers
    - Kubernetes: Orchestrates multiple containers across a cluster, automates scaling, networking, and deployments
2. **Is Kubernetes used in your organization?**
    - Check whether your company has microservices architecture or cloud-native workloads
    - Ask about EKS, self-managed Kubernetes, or other orchestration tools